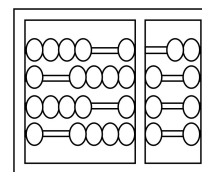




INF0093 - PROJETO
PRÁTICO COM
SISTEMAS
MULTIAGENTES - 2S
2026



PROF. MARCELO DA SILVA
REIS
msreis@unicamp.br

Entregável 2 — Workflow, ReAct, Memória e Integrações

Campinas, 4 de setembro de 2026

Sumário

1	Objetivo	2
2	Formato	2
2.1	Como partir do Entregável 1	2
2.2	Correção do conjunto de avaliação	3
2.3	Estrutura herdada do Entregável 1	3
2.4	Opcional: entrega de sistema executável por linha de comando	3
3	Evolução do baseline	4
3.1	Workflow e/ou ReAct	4
3.2	Contexto e memória	4
3.3	Ferramentas e MCP	5
3.4	Comparação com o baseline	5
3.4.1	Novos modos de falha	6
3.5	Análise arquitetural	6
3.6	Pergunta obrigatória	7
4	Entrega do notebook	7
5	Critérios de avaliação	7

1 Objetivo

O objetivo deste segundo entregável é evoluir o baseline do Entregável 1 para uma versão agêntica com fluxo de controle explícito, uso de ferramentas e gerenciamento de contexto. O grupo deve justificar cada incremento arquitetural a partir das limitações observadas na primeira versão. Nesta etapa, o foco não é apenas “adicionar LangGraph” ou “adicionar ferramentas”. O foco é demonstrar que a nova arquitetura resolve alguma limitação real do baseline ou testa uma hipótese arquitetural clara.

2 Formato

- Grupos de 1–3 estudantes, mantendo a mesma composição do Entregável 1.
- Entrega em notebook Jupyter (.ipynb), com saídas salvas.
- Documentação em Markdown e código em Python.
- Código executável, com instruções de configuração.
- O notebook deve conter a comparação com o baseline do Entregável 1.

É recomendável manter o versionamento em repositório git. Apenas um membro do grupo deve fazer a submissão no Classroom.

2.1 Como partir do Entregável 1

1. Renomeie o arquivo de template do Entregável 2 para `E2_sobrenomes.ipynb`;
2. Consolide, logo no início, um bloco de **estrutura herdada do Entregável 1**, reunindo documento ou entradas de exemplo, esquema de saída, conjunto de casos congelado, funções de verificação e o registro da execução;
3. Acrescente, depois desse bloco, as seções novas exigidas por este entregável, seguindo o roteiro do template do Entregável 2.

O notebook da Aula 3 é um exemplo do uso da estrutura herdada do baseline. Esta convenção vale para os demais entregáveis: **cada entrega é um notebook novo**, que abre com o bloco de estrutura herdada e traz apenas a arquitetura daquela versão.

2.2 Correção do conjunto de avaliação

Como regra, os casos do Entregável 1 não podem ser modificados. Abre-se uma exceção quando o conjunto tiver um defeito real; por exemplo, caso mal formulado, resposta de referência incorreta ou verificação que não mede o que pretendia. Nesse caso, o grupo pode corrigi-lo, desde que: (i) registre no notebook o que foi alterado e por quê; e (ii) **reexecute o baseline sobre o conjunto corrigido**, de modo que a comparação continue usando a mesma régua. O suplemento traz uma função que gera uma impressão digital do conjunto, permitindo verificar entre entregas se ele mudou.

2.3 Estrutura herdada do Entregável 1

Devem ser reaproveitados sem alteração:

- o conjunto de casos congelado (novos casos podem ser *acrescentados*; os antigos não podem ser modificados nem removidos);
- as funções de verificação e os critérios de sucesso;
- o formato de saída, para que as respostas das duas versões sejam comparáveis;
- o registro da execução (modelo, temperatura, versão do prompt, data).

O baseline deve ser **reexecutado nesta entrega**, com o mesmo modelo e no mesmo ambiente da v2. Comparar a v2 de hoje com números coletados há uma semana introduz uma variável que o grupo não controla. Pelo mesmo motivo, **não troque de modelo entre v1 e v2**: se o modelo mudar junto com a arquitetura, não será possível atribuir a diferença observada a nenhum dos dois.

2.4 Opcional: entrega de sistema executável por linha de comando

Se o sistema que você implementa necessita (ou necessitará nas próximas etapas) de recursos que inviabilizam o uso de Jupyter notebook ou torna este último não desejável para os seus objetivos na disciplina (*e.g.*, um posterior *deploy* do sistema), pode-se adicionalmente entregar um arquivo **zip** contendo o código-fonte executável por linha de comando. Nesse caso, os seguintes requisitos devem ser atendidos:

- O **zip** deve conter um README que explique claramente como executar por linha de comando cada experimento que deveria estar em uma das células de código no Jupyter notebook;
- O código deve prover a instalação e a configuração de todas as dependências externas. Caso isso não seja possível, você deve mockar o funcionamento dessas dependências;
- Além do arquivo **zip**, o grupo deve enviar também o arquivo **.ipynb** com os Markdowns preenchidos como se tivesse feito uma entrega com o sistema e os experimentos preenchidos nas células de código.

3 Evolução do baseline

3.1 Workflow e/ou ReAct

A v2 deve ser implementada em LangGraph e conter fluxo de controle explícito. A solução deve incluir:

- estado explícito;
- pelo menos um fluxo com mais de uma etapa;
- pelo menos uma ferramenta útil;
- roteamento condicional, seleção de ferramenta ou outro mecanismo de decisão;
- condição clara de término, incluindo um limite explícito de passos (por exemplo, `recursion_limit`).

Um laço agente-ferramentas pode não terminar sozinho, e sem limite o custo é contido apenas pela cota da API. Se o limite for atingido durante os testes, isso deve ser registrado como modo de falha. Além disso, as ferramentas implementadas devem **fazer trabalho real**: uma função que devolve sempre a mesma resposta fixa, independentemente da entrada, não caracteriza uso de ferramenta e invalida a comparação com o baseline.

O grupo deve justificar se a solução é predominantemente:

- um workflow determinístico;
- um agente ReAct;
- ou uma arquitetura híbrida.

A justificativa deve considerar o problema, não apenas a tecnologia.

3.2 Contexto e memória

A versão deve demonstrar como o sistema gerencia contexto. Quando aplicável, implemente memória de curto prazo ou checkpointing e mostre pelo menos um caso em que uma interação posterior depende do contexto anterior.

Quando houver memória conversacional, recomenda-se **demonstrar primeiro a falha sem ela**: a mesma interação executada sem contexto anterior, mostrando o que o sistema erra ou deixa de responder. Essa saída é a evidência que justifica o incremento. Caso memória conversacional não seja pertinente ao problema, o grupo deve explicar qual estado precisa ser preservado durante a execução e por quê. Exemplos: documento em análise, identificador de tarefa, resultados parciais, plano de execução, chamadas a ferramentas ou preferências relevantes.

Dois pontos adicionais, quando aplicáveis:

- **Custo do contexto:** o histórico reenviado a cada turno faz o custo crescer. Se o sistema mantiver conversas longas, meça esse crescimento e indique a estratégia de contenção adotada (recorte, resumo ou recuperação sob demanda).
- **Isolamento:** se houver mais de um usuário ou mais de uma conversa simultânea, explique como as conversas são mantidas separadas.

3.3 Ferramentas e MCP

Identifique pelo menos uma capacidade externa relevante ao projeto. O grupo deve:

1. descrever a integração;
2. explicar se ela foi implementada como ferramenta local, via MCP ou por outra interface;
3. justificar a decisão arquitetural;
4. demonstrar a integração no notebook, quando viável.

Ao integrar fontes externas, lembre-se de que o retorno de uma ferramenta entra no contexto do modelo e deve ser tratado como **entrada não confiável**, e não como instrução. Documentos e serviços externos podem conter tentativas de injeção de prompt. Recomenda-se conceder o menor privilégio necessário à ferramenta e registrar as chamadas realizadas com seus argumentos.

O uso de MCP **não é obrigatório quando não houver justificativa arquitetural**. Nesses casos, o grupo deve explicar por que uma ferramenta local ou integração direta é mais apropriada nesta versão. Por outro lado, se o projeto envolver recursos que seriam reutilizados por múltiplos agentes, aplicações ou workflows, MCP deve ser considerado explicitamente.

3.4 Comparação com o baseline

Reexecute os casos de teste do Entregável 1. Se necessário, adicione novos casos para capturar capacidades da v2, mas a comparação principal deve usar a mesma régua do baseline.

Compare baseline e v2 considerando pelo menos:

- qualidade/correção;
- capacidade de resolver tarefas compostas;
- comportamento diante de informação ausente;
- número de chamadas ao LLM;
- número de chamadas a ferramentas;
- latência aproximada;

- novos modos de falha.

Todos os resultados devem ser registrados no notebook. Quando houver avaliação manual, a rubrica deve ser explicitada.

Ao interpretar os números, considere o tamanho do conjunto: com poucos casos, uma diferença de um acerto representa uma variação percentual grande e não caracteriza tendência. Prefira afirmações do tipo “a v2 acertou o caso composto que a v1 errava” a afirmações genéricas como “a v2 é melhor”.

Uma v2 que empata ou piora em relação ao baseline é um resultado válido, desde que relatada com hipótese sobre a causa. O que se avalia é a qualidade da investigação, não o desempenho da versão mais recente.

3.4.1 Novos modos de falha

O desenvolvimento do v2 introduz falhas que o baseline não tinha. Verifique e relate, quando ocorrerem:

- ferramenta correta chamada com argumento errado;
- ferramenta chamada sem necessidade (custo sem ganho);
- ferramenta necessária não chamada;
- erro dentro da ferramenta tratado silenciosamente;
- laço interrompido pelo limite de passos;
- resposta final que ignora o que a ferramenta devolveu.

3.5 Análise arquitetural

Discuta:

- que limitações do baseline foram resolvidas;
- que limitações permaneceram;
- que novas limitações surgiram;
- quais partes do sistema continuam acopladas;
- quais responsabilidades poderiam ser especializadas;
- o que poderia justificar a transformação da v2 em um sistema multiagente.

3.6 Pergunta obrigatória

Ao final do notebook, responda:

Que responsabilidade do sistema atual seria a melhor candidata a se tornar um agente especializado na próxima versão, e por quê?

A resposta a essa pergunta será usada como ponto de partida para o Entregável 3.

4 Entrega do notebook

A entrega do `.ipynb` deverá ser feita até dia **13 de setembro**, utilizando o Google Classroom da disciplina. O notebook deve ser executado do início ao fim e salvo com as saídas visíveis.

5 Critérios de avaliação

Critério	Peso
Justificativa da evolução em relação ao baseline	20%
Workflow/ReAct e uso adequado de LangGraph	25%
Contexto, memória e integrações	20%
Comparação experimental com o baseline	20%
Análise arquitetural, documentação e reprodutibilidade	15%